Rúbrica Evaluación: Sumativa N°3
Programación Orientada a Objeto Seguro
Unidades de aprendizaje 2 y 3: Software orientado a objetos, persistencia y seguridad en Python (ponderación de evaluación 35 %)

Área Académica	Tecnologías de Información y Ciberseguridad	**Carrera**	Ingeniería Informática
Sede	Puente Alto	**Código**	TI3V21
Docente	Michael Arjel	**Fecha**	Martes 13 de octubre de 2026
Sección	114-2B-F1	**Duración**	1 semana (6 al 13 de octubre)

Nombre Equipo:			
Negocio asignado:			
Integrante 1:			
	Apellido Paterno	Apellido Materno	Nombres
Integrante 2:			
	Apellido Paterno	Apellido Materno	Nombres
Rut integrantes:			

Puntaje Máximo	42	**Nota:**		**Firma Conforme**	
Puntaje Obtenido					
Solicita Re-Corrección	Sí	No	**Motivo:**		

INSTRUCCIONES GENERALES:

- La nota 4.0 se obtiene logrando un 60% del puntaje total, es decir 25 de 42 puntos.
- La entrega es digital: el enlace al repositorio GitHub del equipo, en el Ambiente de Aprendizaje INACAP.
- El plazo vence el martes 13 de octubre a las 23:59. Se evalúa el último commit anterior al plazo.
- El repositorio debe ser público. Un repositorio privado o un enlace que no abre se considera no entregado.
- El docente ejecuta el programa siguiendo solo las instrucciones del README. Lo que no funcione así se evalúa como no funcionando.
- La nota es la misma para todo el equipo.
- Puede usar asistentes de inteligencia artificial, pero debe documentar en el README qué adoptó, modificó o descartó, y por qué.

Aprendizajes esperados:

2.1.- Construye una solución de software en Python para resolver una problemática planteada, considerando el paradigma de programación orientada a objetos, la conexión segura a bases de datos y la aplicación de criterios técnicos para validar e integrar recomendaciones generadas por herramientas de IA.
3.1.- Desarrolla scripts en Python para consumir servicios externos y persistir datos localmente, aplicando mecanismos de seguridad de la información y considerando herramientas de IA como apoyo al desarrollo.

Criterios de evaluación:

2.1.2.- Aplica principios de programación orientada a objetos en la solución desarrollada en Python, con coherencia con el modelo diseñado y sin automatización del código.
2.1.3.- Emplea librerías oficiales de Python para la conexión a bases de datos, según el sistema gestor seleccionado y los requerimientos de la solución.
3.1.1.- Programa scripts en Python en el consumo de servicios externos, utilizando librerías oficiales.
3.1.2.- Utiliza mecanismos de seguridad en el acceso y procesamiento de datos mediante técnicas de autenticación y validación de entradas en los scripts desarrollados.
3.1.3.- Aplica mecanismos de manejo de errores en el consumo de servicios externos, mediante estructuras de control que permitan la continuidad y estabilidad del sistema.
3.1.4.- Ajusta la seguridad del código desarrollado con apoyo de herramientas de IA, documentando y justificando las decisiones técnicas adoptadas.

Presentación de la actividad

En la Evaluación Sumativa N°2 entregaron el modelo de clases de su negocio en Python, con su informe, y recibieron correcciones. Esta evaluación completa el sistema sobre ese mismo repositorio: los datos se guardan en una base de datos, las entradas del usuario se validan, y el precio que depende de un indicador externo se calcula con el valor real obtenido desde una API.

Es una actividad grupal, con los mismos equipos y el mismo negocio de las evaluaciones anteriores, y corresponde al 35 % de la nota del módulo. No tiene presentación: se evalúa el repositorio tal como queda al cierre del plazo. Incorporen también las correcciones que recibieron en la Evaluación Sumativa N°2.

Al terminar, el programa debe cubrir los cinco requisitos mínimos de la ficha del negocio: tres subtipos con un método que se comporta distinto en cada uno, al menos un dato con validación, una transacción con líneas de detalle, dos reglas que impidan una operación, y un precio que dependa de un indicador externo.

Actividades

Actividad N°1: Persistencia en base de datos

- Base de datos SQLite, con una carpeta `dao/` separada de `model/`: un DAO por cada entidad principal del negocio.
- Operaciones CRUD completas (crear, leer, actualizar y eliminar) sobre las entidades principales.
- La transacción se guarda junto con sus líneas de detalle.
- Todas las consultas parametrizadas. Ninguna consulta se arma concatenando texto que escribe el usuario.
- El programa crea las tablas si no existen, de modo que corra en un computador nuevo.

Actividad N°2: Validación de entradas

- Las entradas del usuario se validan antes de usarse: tipo, formato y rango.
- El dato que la ficha exige validar se valida en el setter de su clase.
- Un dato inválido se rechaza con un mensaje y el programa sigue funcionando.

Actividad N°3: Consumo de API

- El precio que depende de un indicador externo obtiene su valor desde una API pública, con la librería `requests`. Por ejemplo, `mindicador.cl` entrega el dólar y la UF del día.
- La consulta a la API queda en una función o clase propia, no repartida por el código.
- La consulta define un tiempo máximo de espera. Si la API no responde, tarda demasiado o devuelve algo inesperado, el programa lo informa y sigue funcionando.

Actividad N°4: README del repositorio

El archivo `README.md` debe contener, en este orden:

1. Integrantes y negocio.
2. Cómo instalar y ejecutar el programa, paso a paso, desde un computador que solo tiene Python.
3. Decisiones de seguridad: cómo se evita la inyección SQL y qué entradas se validan.
4. Uso de inteligencia artificial: al menos un ejemplo concreto de código sugerido por la IA que adoptaron, modificaron o descartaron, con la razón técnica.

Requerimiento de presentación y entrega

Formato	Repositorio GitHub público, con `model/`, `dao/`, `main.py`, `README.md` y `requirements.txt` en la raíz.
Medio de entrega	Enlace al repositorio en el Ambiente de Aprendizaje INACAP (AAI), en la tarea "Evaluación Sumativa N°3". Un enlace por equipo.
Plazo	Martes 13 de octubre de 2026, 23:59 horas. Los commits posteriores no se consideran.
Ejecución	El docente clona el repositorio y ejecuta el programa siguiendo solo el README.
Equipos	Los mismos de las evaluaciones anteriores. La nota es la misma para todo el equipo.

Instrumento de evaluación

Para verificar lo que han aprendido, luego de desarrollar esta evaluación, revise el repositorio de sus estudiantes a través de la siguiente rúbrica.

Criterio	Dimensión /Indicadores	Niveles de desempeño.					Observaciones
		Destacado (7 puntos)	Habilitado (5 puntos)	En desarrollo (3 puntos)	No logrado (1 punto)	Puntaje obtenido	
2.1.2 Aplica principios de programación orientada a objetos, con coherencia con el modelo diseñado	Integración del sistema y cobertura de los cinco requisitos	El programa corre de principio a fin siguiendo el README y cubre los cinco requisitos. Las operaciones usan las clases del modelo, no diccionarios ni listas sueltas.	Cubre cuatro de los cinco requisitos, o hay un error no controlado en una operación secundaria.	Cubre dos o tres requisitos, o el programa se cae en la operación principal del negocio.	Cubre menos de dos requisitos, o no se puede ejecutar siguiendo el README.		
2.1.3 Emplea librerías oficiales de Python para la conexión a bases de datos	Persistencia con DAO y consultas parametrizadas	CRUD completo de las entidades principales mediante DAO separados del modelo. La transacción se guarda con sus líneas de detalle. Todas las consultas son parametrizadas y las tablas se crean al iniciar.	CRUD completo, pero una entidad principal queda con operaciones parciales, o hay SQL dentro de una clase del modelo.	Solo algunas operaciones (por ejemplo, insertar y listar), o alguna consulta concatena texto que escribe el usuario.	No hay base de datos: los datos viven solo en memoria.		
3.1.2 Utiliza mecanismos de seguridad mediante validación de entradas	Validación de entradas	Todas las entradas del usuario se validan antes de usarse (tipo, formato y rango). El dato que exige la ficha se valida en el setter, y un dato inválido se rechaza con un mensaje sin detener el programa.	Se validan las entradas, salvo alguna que llega sin validar.	Solo se valida el dato que exige la ficha; el resto de las entradas se usa sin validar.	No hay validación: un dato inválido se guarda o hace caer el programa.		

Criterio	Dimensión /Indicadores	Niveles de desempeño.					Observaciones
		Destacado (7 puntos)	Habilitado (5 puntos)	En desarrollo (3 puntos)	No logrado (1 punto)	Puntaje obtenido	
3.1.1 Programa scripts en Python en el consumo de servicios externos	Precio calculado con un indicador obtenido desde una API	El precio usa el valor del indicador obtenido desde una API pública con `requests`. La consulta está en una función o clase propia.	Consume la API y usa el valor en el precio, pero la consulta está repartida en varias partes del código.	Consume la API, pero el valor solo se muestra y no se usa en el cálculo del precio.	El valor del indicador está escrito a mano en el código, o no se consume ninguna API.		
3.1.3 Aplica mecanismos de manejo de errores en el consumo de servicios externos	Continuidad del sistema cuando la API falla	Define un tiempo máximo de espera. Sin conexión, con demora o con una respuesta inesperada, el programa informa el problema y sigue funcionando.	Maneja la falta de conexión, pero no la demora o la respuesta inesperada.	Captura el error con un `except` genérico que lo oculta sin informar.	El programa se cae si la API no responde.		
3.1.4 Ajusta la seguridad del código con apoyo de IA, documentando y justificando las decisiones	README con decisiones de seguridad y uso de IA	El README trae los cuatro apartados pedidos. Las decisiones de seguridad están explicadas y el uso de IA incluye un ejemplo concreto con su razón técnica.	Trae los cuatro apartados, pero las justificaciones son generales, sin ejemplos concretos.	El README solo explica cómo ejecutar el programa.	No hay README, o sus instrucciones no permiten ejecutar el programa.		
	Puntaje total						**/ 42**

Tabla de puntaje y nota

Escala con 60 % de exigencia sobre 42 puntos totales. La nota 4.0 se alcanza con 25 puntos, destacada en la tabla.

Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota
0	1.0	11	2.3	22	3.6	33	5.4
1	1.1	12	2.4	23	3.8	34	5.6
2	1.2	13	2.6	24	3.9	35	5.8
3	1.4	14	2.7	**25**	**4.0**	36	5.9
4	1.5	15	2.8	26	4.2	37	6.1
5	1.6	16	2.9	27	4.4	38	6.3
6	1.7	17	3.0	28	4.5	39	6.5
7	1.8	18	3.2	29	4.7	40	6.6
8	2.0	19	3.3	30	4.9	41	6.8
9	2.1	20	3.4	31	5.1	42	7.0
10	2.2	21	3.5	32	5.2		